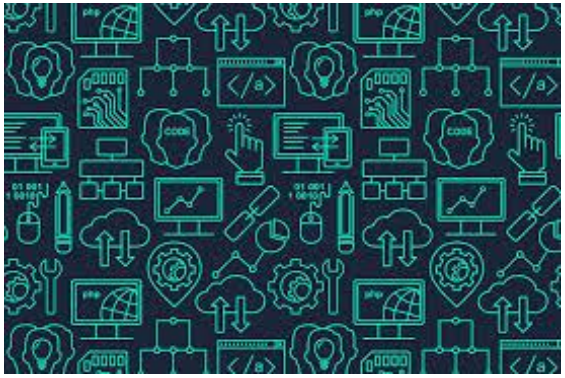
INF 1001
INTRODUCTION TO COMPUTING

Number Systems

Lecture 2

Prof. Vinod Prasad
Email: vinod.prasad@singaporetech.edu.sg



Outline

- Octal and Hexadecimal Number System
- Computer Arithmetic
- Conversion of Number Systems
- Fixed and Floating Point
- Binary Coded Decimal
- Gray Code, ASCII

Octal Number System

- Octal digits: $D = \{0, 1, 2, \ldots, 7\}$
- Number base is 8
- Octal number representation:
  - $a_n a_{n-1} \ldots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \ldots a_{-m}$
  - where the coefficients, $a_i \in D$

Octal point

- Value:

$$a_n \times 8^n + a_{n-1} \times 8^{n-1} + \cdots + a_3 \times 8^3 + a_2 \times 8^2 + a_1 \times 8^1 + a_0 \times 8^0 + a_{-1} \times 8^{-1} + a_{-2} \times 8^{-2} + \cdots + a_{-m} \times 8^{-m}$$

$$= \sum_{i=-m}^{n} a_i \times 8^i$$

Example: (207)8

$(207)_8 = 7 \times 8^0 + 0 \times 8^1 + 2 \times 8^2 = 7 + 0 + 128 = (135)_{10}$

(also written as 135_{10})

Hexadecimal Number System

- Hexadecimal digits: $D = \{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$
- Number base is 16
- Hexadecimal representation: Hexadecimal point
  - $a_n a_{n-1} \dots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \dots a_{-m}$
  - where the coefficients, $a_i \in D$
- Value:

A=10
B=11
C=12
D=13
E=14
F=15

$$a_n \times 16^n + a_{n-1} \times 16^{n-1} + \cdots + a_3 \times 16^3 + a_2 \times 16^2 + a_1 \times 16^1 + a_0 \times 16^0 + a_{-1} \times 16^{-1} + a_{-2} \times 16^{-2} + \cdots + a_{-m} \times 16^{-m}$$

$$= \sum_{i=-m}^{n} a_i \times 16^i$$

Hexadecimal number is also denoted using the suffix 'H' or 'h'.

(2C6B)16 = (2C6B)H = (2C6B)h

Example: Find the decimal equivalent of $(352)_{16}$

$(352)_{16} = 2\text{x}16^0 + 5\text{x}16^1 + 3\text{x}16^2 = 2 + 80 + 768 = 850_{10}$.

Hexadecimal Number System

Example: Find the decimal equivalent of $(12A)_{16}$

$(12A)_{16} = 10x16^0 + 2x16^1 + 1x16^2 = 10 + 32 + 256 = 298_{10}$.

Example: Find the decimal equivalent of $(ABC)_{16}$

$(ABC)_{16} = 12x16^0 + 11x16^1 + 10x16^2 = 12 + 176 + 2560 = 2748_{10}$.

Example: Find the decimal equivalent of $(1010EF)_{16}$

$$(1010EF)_{16} = 15x16^0 + 14x16^1 + 0x16^2 + 1x16^3 + 0x16^4 + 1x16^5$$
$$= 15 + 224 + 0 + 4096 + 0 + 1048576$$
$$=1{,}052{,}911_{10}.$$

Note: In Hex, A=10, B=11, C=12, D=13, E=14, F=15

Number system

- What is the advantage of having a number system with a larger base?

Numbers with Different Bases

Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hexadecimal (base 16)
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

We need only fewer digits to represent a number when the base is larger (compact form).

Example: $(100)_{10}$

$= (1100100)_2 \rightarrow$ 7 bits

$= (144)_8 \rightarrow$ 3 digits

$= (64)_{16} \rightarrow$ 2 digits

Number base Conversions

Forward Conversion: Base-*r* to Decimal

The conversion of a number in base-*r* to decimal is done by expanding the number in a power series and adding all the terms.

Example: Convert $(101101)_2$ to decimal.

Ans: $1x2^0+1x2^2+1x2^3+1x2^5 = (45)_{10}$

Example: Convert $(10110.101)_2$ to decimal.

Ans: $1x2^{-3}+1x2^{-1}+1x2^1+1x2^2+1x2^4 = (22.625)_{10}$

Number base Conversions

Reverse Conversion: Integer Decimal to Base-*r*

The conversion of a decimal integer to a number in base *r* is done by dividing the number and all successive quotients by *r* and accumulating the remainders.

Example: Convert $(41)_{10}$ to binary.

Remainder

Divide by base

2	41	**1**
2	20	**0**
2	10	**0**
2	5	**1**
2	2	**0**
	1	

Quotient

Ans: $(101001)_2$

Number base Conversions

Reverse Conversion: Fractional Decimal to Base-*r*

Example: Convert $(0.6875)_{10}$ to binary.

Procedure: First, 0.6875 is multiplied by 2 to give an integer and a fraction. Then the new fraction is multiplied by 2 to give a new integer and a new fraction. The process is continued until the fraction becomes 0 or until the number of digits has sufficient accuracy.

	Integer		Fraction	Coefficient
$0.6875 \times 2 =$	1	+	0.3750	$a_{-1} = 1$
$0.3750 \times 2 =$	0	+	0.7500	$a_{-2} = 0$
$0.7500 \times 2 =$	1	+	0.5000	$a_{-3} = 1$
$0.5000 \times 2 =$	1	+	0.0000	$a_{-4} = 1$

Therefore, $(0.6875)_{10} = (0.1011)_2$

Number base Conversions

Example: Convert $(123)_8$ to decimal.

Ans: $1x8^2 + 2x8^1 + 3x8^0 = (83)_{10}$

Example: Convert $(41.6875)_{10}$ to binary.

$(41.6875)_{10} = (101001.1011)_2$

Conversion from binary to octal/hexadecimal

Conversion from and to binary, octal, and hexadecimal plays an important role in digital computers, because shorter patterns of hex characters are easier to recognize than long patterns of 1's and 0's.

Conversion from Binary to Octal:

Partition the binary number into groups of 3 digits each (because $2^3 = 8$), starting from the binary point and proceeding to the left and to the right. The corresponding octal digit is then assigned to each group.

<u>Example</u>: Convert $(10110001101011.111100000110)_2$ to octal.

Ans: Group into 3 bits from binary point:

$$(10\ 110\ 001\ 101\ 011\ .\ 111\ 100\ 000\ 110)_2$$

$$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow$$

$$2 \quad 6 \quad 1 \quad 5 \quad 3 \quad 7 \quad 4 \quad 0 \quad 6 \quad = (26153.7406)_8$$

Conversion from binary to hexadecimal

Partition the binary number into groups of 4 digits each (because $2^4 = 16$), starting from the binary point and proceeding to the left and to the right. The corresponding hex digit is then assigned to each group.

<u>Example:</u> $(10110001101011.1111\ 0010)_2$ to hexadecimal.

Ans: Group into 4 bits from binary point:

$$(10\ \ 1100\ \ 0110\ \ 1011\ .\ 1111\ \ 0010)$$

$$\downarrow \qquad \downarrow \qquad \downarrow \qquad \downarrow \qquad \downarrow \qquad \downarrow$$

$$2 \qquad C \qquad 6 \qquad B \qquad F \qquad 2$$

$$(= 2C6B.F2)_{16}$$

Note: In Hex, A=10, B=11, C=12, D=13, E=14, F=15

Binary Arithmetic

Binary Addition

A + B	SUM	CARRY
0 + 0	0	0
0 + 1	1	0
1 + 0	1	0
1 + 1	0	1

```
      1  1  1  1  ←——— carry
      1  1  1  0  1      =29
(+)   1  1  0  1  1      =27
-----------------------
   1  1  1  0  0  0      =56
-----------------------
```

Binary Subtraction

Minuend	Subtrahend	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

```
                    0 0 1 1 1 1 1 0 0  Borrow
X     229             1 1 1 0 0 1 0 1
Y  -   46     -       0 0 1 0 1 1 1 0
      183             1 0 1 1 0 1 1 1
```

Binary Addition

Example: $00101001_2 + 00011101_2$

8-bit unsigned binary

	128	64	32	16	8	4	2	1	
	0	0	1	0	1	0	0	1	(41 in decimal)
+	0	0	0	1	1	1	0	1	(29 in decimal)

	128	64	32	16	8	4	2	1
	0	0	1	0	1	0	0	1
+	0	0	0	1	1	1	0	1
=	0	1	0	0	0	1	1	0

0+0 =0

1+0+0= 0

1+1+0= 0+C(=1)

1+0+1= 0+C(=1)

1+1= 0+C(=1)

0+1 =1

0+1 =1

1+1= 0+C(=1)

$01000110_2 = 70_{10}$

$41_{10} + 29_{10} = 70_{10}$

Binary Subtraction

Example: 110_2 - 101_2

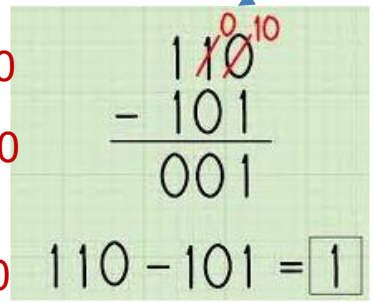
Borrow '1' from adjacent digit

So, $10_2 - 1_2 = 1_2$

6_{10}

5_{10}

$=1_{10}$

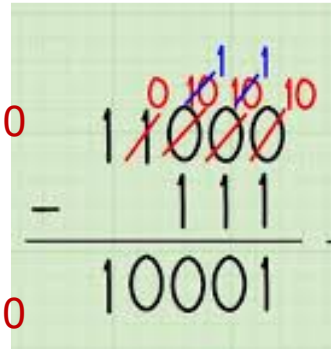


Example: 11000_2 - 111_2

24_{10}

7_{10}

$=17_{10}$



Binary Arithmetic

Binary Multiplication

Example: Find $(1\ 1\ 0)_2 \times (1\ 1)_2$

carry

$$
\begin{array}{r}
1\ 1\ 0 \\
\times\quad 1\ 1 \\
\hline
1\ \ 1\ 1\ 1\ 0 \\
1\ 1\ 0\ \times \\
\hline
1\ 0\ 0\ 1\ 0 \\
\hline
\end{array}
$$

A + B	SUM	CARRY
0 + 0	0	0
0 + 1	1	0
1 + 0	1	0
1 + 1	0	1

Computer Arithmetic for Binary Multiplication & Division

- Shift left by '*x*' units = multiply by '*x*'-th power-of-2

$2_{10} = 0010_2$

$4_{10} = 0100_2 = 0010 <<1$ (i.e., multiply by 2^1)

'<<' → shift left operation

- Shift right by '*x*' units = divide by '*x*'-th power-of-2

$8_{10} = 1000_2$

$4_{10} = 0100_2 = 1000 >>1$ (i.e., divide by 2^1)

'>>' → shift right operation

Left / right Shift operation is performed using SHIFT REGISTERS.

Therefore, division and multiplication by power-of-two is performed by Shift Registers in Digital Systems.

(You will study Shift Registers later).

Computer Arithmetic for Binary Multiplication

$1\ 0\ 1\ 1_2 \times 1\ 0\ 1_2$?

$1\ 0\ 1\ 1_2 = 11_{10}$

$1\ 0\ 1_2 = 5_{10}$

Product = 55_{10}

```
                1  0  1  1 x
                   1  0  1
              ----------------
                1  0  1  1   +
Shift by '1' unit  0  0  0  0   +
Shift by '2' units 1  0  1  1   +
              ----------------
             1  1  0  1  1  1   <- 55_10
```

Multiplication can be done by 'shift' and 'add' operations (faster).

Product of '*M*'-bit number and '*N*'-bit number can be up to '(*M*+*N*)' bits.

What if the register wordlength is only 'K' bits, which is less than '(*M*+*N*)' bits?

Discard (M+N-K) bits → Beware of this!

Binary-Coded Decimal (BCD) Code

Although the binary number system is the most natural system for a computer, human are more accustomed to the decimal.

One way to resolve this difference is to convert decimal numbers to binary, perform all arithmetic calculations in binary, and then convert the binary results back to decimal.

This method requires that we store decimal numbers in the computer so that they can be converted to binary.

Since the computer can accept only binary values, we must represent the decimal digits by means of a code that contains 1's and 0's.

It is also possible to perform the arithmetic operations directly on decimal numbers when they are stored in the computer in coded form.

Binary-Coded Decimal (BCD) Code

A binary code that distinguishes among 10 elements must contain at least four bits, thus total $2^4 = 16$ combinations, but 6 out of the 16 possible combinations remain unassigned.

BCD is a weighted code

The binary combinations 1010 through 1111 are not used and have no meaning in BCD.

Example: $(185)_{10} = (0001\ 1000\ 0101)_{BCD}$

$= (10111001)_2$

Decimal Digit	BCD 8421
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
	1010
Unused	1011
bit	1100
combi-	1101
nations	1110
	1111

More about BCD

BCD number needs more bits than its equivalent binary value.

However, there is an advantage in the use of decimal numbers, because computer input and output data are generated by human who use the decimal system (easier to code decimal in BCD than converting to binary).

BCD numbers are decimal numbers and not binary numbers, although they use bits in their representation.

The only difference between a decimal number and BCD is that decimals are written with the symbols 0, 1, 2, …., 9 and BCD numbers use the binary code 0000, 0001, 0010, …., 1001. The decimal value is exactly the same.

Fixed Point and Floating Point Representation

Limitations of fractional numbers in Binary:

- Exact representation possible only for numbers of the form $x^* 2^y$
- Other rational numbers have repeating bit representations (infinite bits)
  - $1/3 = 0.333333\ldots_{10} = 0.01010101[01]\ldots_2$

Fractional binary numbers can be represented either in fixed point or in floating point form.

Fixed point: Binary point is set in a fixed position, and therefore it does not need to be stored in memory.

Example: 0100.011_2 (binary point is after the 4th bit (from the left). Hence, the fraction is represented in binary as 0100011_2 and this can be interpreted as equivalent to 0100.011_2.

Binary point is fixed my processor designer. Another processor can have the binary point fixed after the 5th bit: 01000.11_2

Floating Point Representation

Analogous to scientific notation

- $12000000 = 1.2 \times 10^{7}$ (=1.2e7)
- $0.0000012 = 1.2 \times 10^{-6}$ (=1.2e-6)

Floating Point = $(-1)^{s} * M * 2^{E}$

- Sign bit ***s*** determines whether number is negative or positive.
- Mantissa ***M*** normally a fractional value in range [1.0,2.0).
- Exponent ***E*** weights value by power-of-two.

Base 10 examples:

If E = 4, M=1.125, then the number = $1.125 \times 10^{4} = 11250$

If E = -2, M=5, then the number = $5 \times 10^{-2} = 0.05$

Base 2 examples:

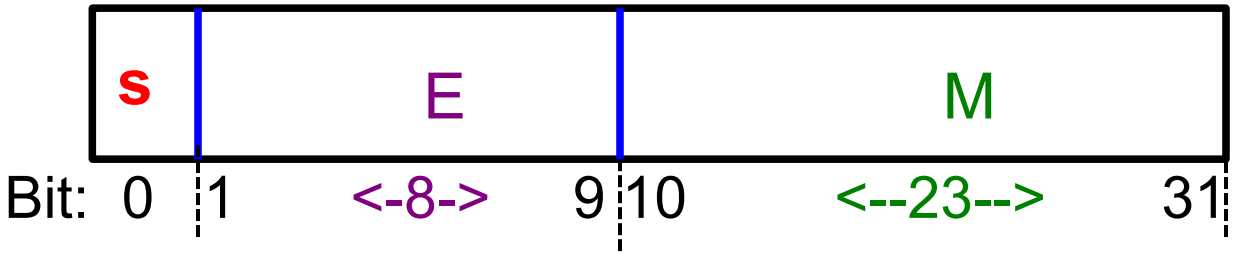
If E= 4, M=1.125, then the number = $1.125 \times 2^{4} = 18$

If E= -2, M=5, then the number = $5 \times 2^{-2} = 1.25$

IEEE 754 Floating Point

IEEE standard 754 floating point representation is used in almost all modern processors.

IEEE754 has 1 sign bit (**s**), 8 exponent bits (*E*) and 23 mantissa bits (*M*) for single-precision numbers.



E and M are both fixed point 2's complement unsigned numbers
Stored together. e.g. 0000 1000 1110 0000 0000 0000 0000 0000

Double precision numbers have 1, 11, and 52 bits for s, E, and M respectively.

IEEE 754 Floating Point

- The exponent is stored in **excess-127** for single precision (and **excess 1023** for double precision).

- It means that you must add 127 to it before storing it (or subtract 127 from it when reading the number).

- The mantissa is also slightly unusual; the 23 (52) bits are in fractional format and it is assumed that 1 must be added to the mantissa, but the 1 is not stored. It is a **hidden 1**. In other words, the mantissa is a number between 1 and 2.

IEEE 754 Floating Point

Example:

0011 1111 1000 0000 0000 0000 0000 0000

sign (1 bit) exponent (8 bits) mantissa (23 bits)

- Sign is 0 so this is positive
- Exponent is 64+32+16+8+4+2+1–127 = 127–127 = 0
  (remember it's in excess-127 format).
- Mantissa is 0, but with the hidden 1, it becomes 1.0.
- FP = $(-1)^s * M * 2^E$
- So the number stored is $1.0 \times 2^0 = 1.0$

IEEE 754 Floating Point

Example:

1100 0010 0010 0000 0000 0000 0000 0000

sign (1 bit) exponent (8 bits) mantissa (23 bits)

- Sign is 1, so this is negative
- Exponent is 128+4=132, but we must subtract 127.
  E = 132-127=5
- Mantissa is 0.25 + the hidden 1 = 1.25
- FP = $(-1)^s * M * 2^E$
- So the number stored is $-1 \times 1.25 \times 2^5 = -40.0$

Gray code

Not an arithmetic code - Only one bit in the code changes, when going from one number to next.

Used in applications in which the normal sequence of binary numbers generated by the hardware may produce an error during the transition from one number to the next.

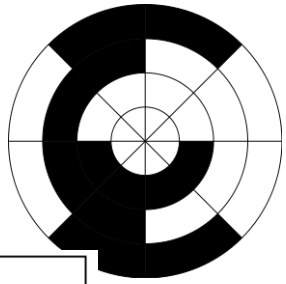
For example, from 0111 to 1000 may produce an intermediate erroneous number 1001 if the value of the rightmost bit takes longer to change than do the values of the other three bits.

Gray Code	Decimal Equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

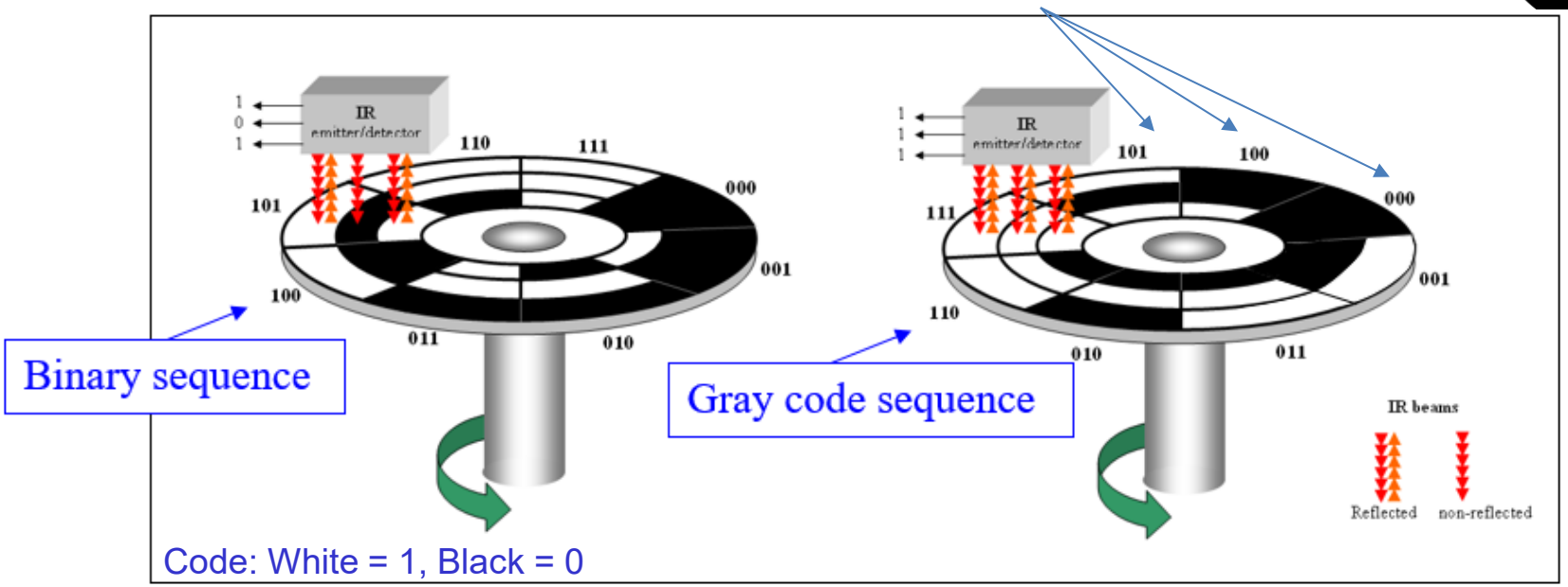
- Useful in analog-to-digital converters (ADCs), Rotary encoders, etc.

Gray code Application – Rotary Encoder

Rotary encoder for angle-measuring devices marked in 3-bit binary.

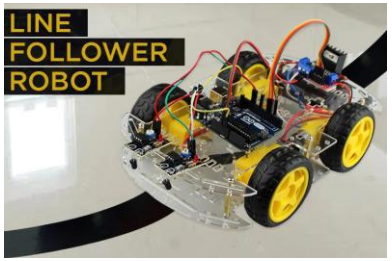


Only one bit in the gray code changes, when going from one number to next, during the rotation of the encoder.



Infrared (IR) emitter emits light. IR light is absorbed by black surface, hence the receiver does not receive any reflected light (=0). IR is reflected by white surface, hence the received receives reflected light (=1)

This principle is used in the line follower robot as well!



American Standard Code for Information Interchange (ASCII)

Many applications of digital computers require not only numbers but also other characters or symbols. **ASCII** is a character encoding standard for electronic communication (computers, communication equipment, etc.).

	$b_7b_6b_5$							
$b_4b_3b_2b_1$	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	“	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	‘	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	|
1101	CR	GS	–	=	M	]	m	}
1110	SO	RS	.	>	N	∧	n	~
1111	SI	US	/	?	O	–	o	DEL

Summary

- Number System - Octal, Hexadecimal
- Conversion of numbers from one number system to the other
- Binary addition, subtraction, multiplication, division
- Binary Coded Decimal
- Fixed Point and Floating Point
- Gray Code
- ASCII